

Guía de CI/CD: Despliegue Automático en Hostinger desde GitHub

Parte 1: Proceso de Configuración

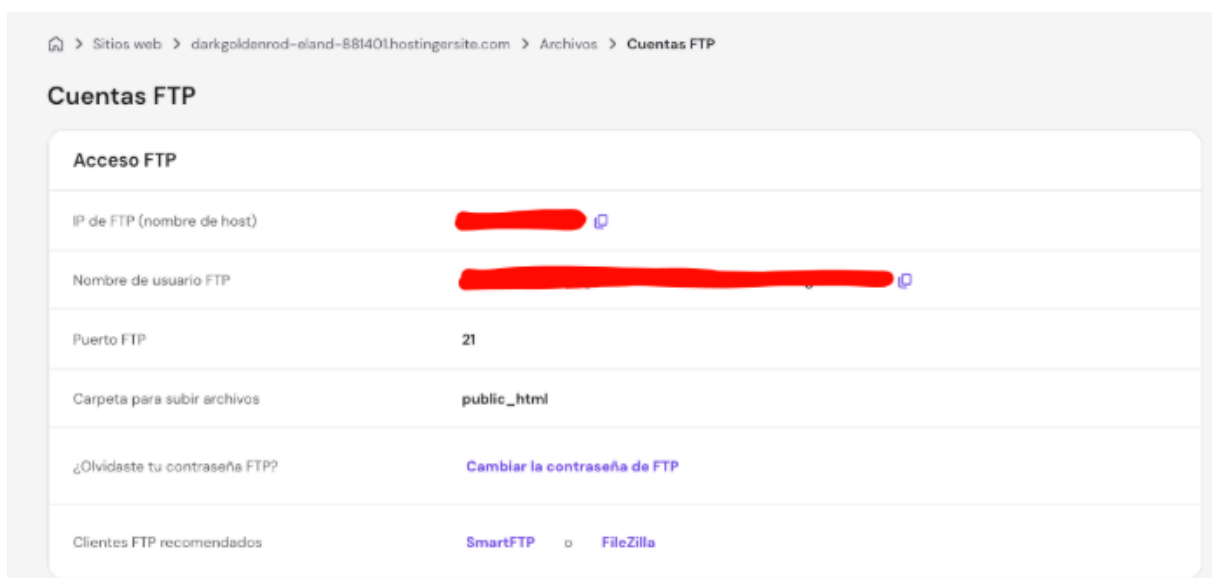
1. Obtener credenciales de Hostinger

Para automatizar el despliegue, necesitamos las llaves de acceso al servidor. Estas las podemos encontrar en el panel del mismo, en mi caso que uso Hostinger sería en el hPanel. Lo que necesitamos es:

- **Credenciales de Base de Datos:** Host, Nombre de la base de datos, Usuario y Contraseña.



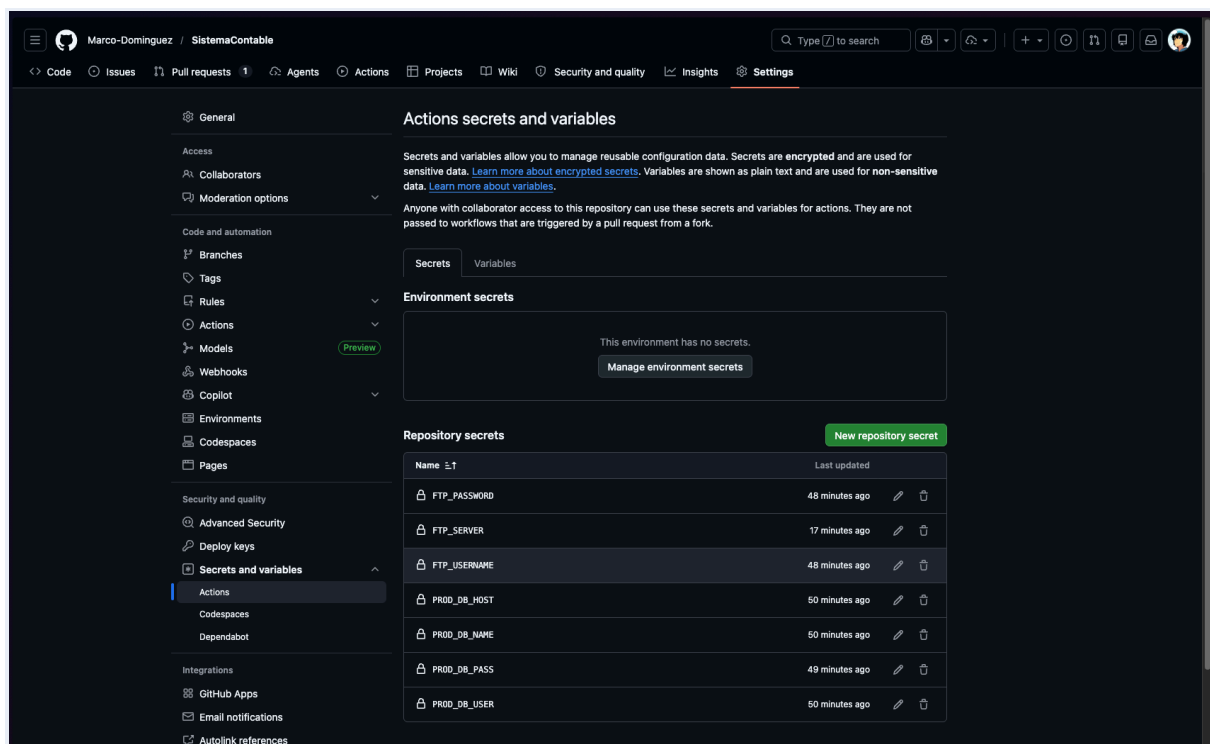
- **Credenciales FTP:** IP/Host del servidor FTP (ej. `ftp.tudominio.com`), Usuario y Contraseña.



2. Configurar "Secrets" en GitHub

Por seguridad, nunca debemos poner contraseñas directamente en el código. Por eso, usaremos las variables secretas de GitHub, para configurarlas hay que hacer lo siguiente:

1. Ir al repositorio en GitHub > **Settings** > **Secrets and variables** > **Actions**.
2. Para agregar una nueva hay que hacer clic en **New repository secret** y registrar las variables con los datos del paso anterior:
 - Para la Base de Datos:
 - i. `PROD_DB_HOST`
 - ii. `PROD_DB_NAME`
 - iii. `PROD_DB_USER`
 - iv. `PROD_DB_PASS`
 - Para el FTP:
 - i. `FTP_SERVER`
 - ii. `FTP_USERNAME`
 - iii. `FTP_PASSWORD`



3. Configurar el archivo `.yaml` de GitHub Actions

En el proyecto local, hay que crear la ruta `.github/workflows/` y dentro un archivo llamado `deploy.yaml` con el código para el deploy, en mi caso este:

```

.github > workflows > Y deploy.yml
1  name: Deploy to Hostinger
2
3  on:
4    push:
5      branches:
6        - main # Se ejecuta cuando hay push a la rama main
7
8  jobs:
9    deploy:
10     runs-on: ubuntu-latest
11
12     steps:
13       # obtener el código fuente
14       - name: Checkout Code
15         uses: actions/checkout@v4
16
17       # reemplazar credenciales de BD en el archivo database.php
18       - name: Update Database Credentials
19         run: |
20           sed -i "s/define('DB_HOST'.*/define('DB_HOST', '{{ secrets.PROD_DB_HOST }}');/" service/config/database.php
21           sed -i "s/define('DB_NAME'.*/define('DB_NAME', '{{ secrets.PROD_DB_NAME }}');/" service/config/database.php
22           sed -i "s/define('DB_USER'.*/define('DB_USER', '{{ secrets.PROD_DB_USER }}');/" service/config/database.php
23           sed -i "s/define('DB_PASS'.*/define('DB_PASS', '{{ secrets.PROD_DB_PASS }}');/" service/config/database.php
24
25       # 3. subir archivos por FTP a Hostinger
26       - name: Deploy FTP
27         uses: SamKirkland/FTP-Deploy-Action@v4.3.5
28         with:
29           server: {{ secrets.FTP_SERVER }}
30           username: {{ secrets.FTP_USERNAME }}
31           password: {{ secrets.FTP_PASSWORD }}
32           # dir default en hostinger
33           server-dir: /public_html/
34
35           exclude: |
36             **/.git*
37             **/.git*/**
38             .github/**
39             README.md
40             assets/**

```

¿Para qué sirve y cómo funciona el archivo `deploy.yml`?

Este archivo es un **flujo de trabajo (workflow)** de GitHub Actions. Actúa como una lista de instrucciones que un servidor virtual de GitHub ejecutará paso a paso:

- **on: push y branches: [main]:** Este le dice a GitHub que estas instrucciones solo deben ejecutarse cuando se suben cambios a la rama 'main'.
- **runs-on: ubuntu-latest:** Define que GitHub prestará una máquina virtual con sistema operativo Linux (Ubuntu) para realizar el trabajo.
- **Paso de "Checkout Code":** Usa una herramienta (`actions/checkout`) para que la máquina virtual descargue el código de la rama más reciente, el que acabamos de subir.
- **Paso de "Update Database Credentials":** Aquí usamos el comando de Linux `sed`. Su función es buscar en el archivo que definamos, en mi caso 'database.php' las variables locales (como `localhost` o `root`) y reemplazarlas por las credenciales que guardamos en los 'Secrets'. Esto lo hago de esta manera debido a que en el código están solo las variables locales y no debemos tener nunca las variables de nuestro servidor en el repositorio de github.
NOTA: este paso es un paso personalizado específicamente para mi proyecto, no es un paso que se configure en todos los proyectos.
- **Paso de "Deploy FTP":** Utiliza una acción llamada `FTP-Deploy-Action` para conectarse a la cuenta de Hostinger usando las credenciales en los 'Secrets'. Compara qué archivos cambiaron y los sube automáticamente al directorio

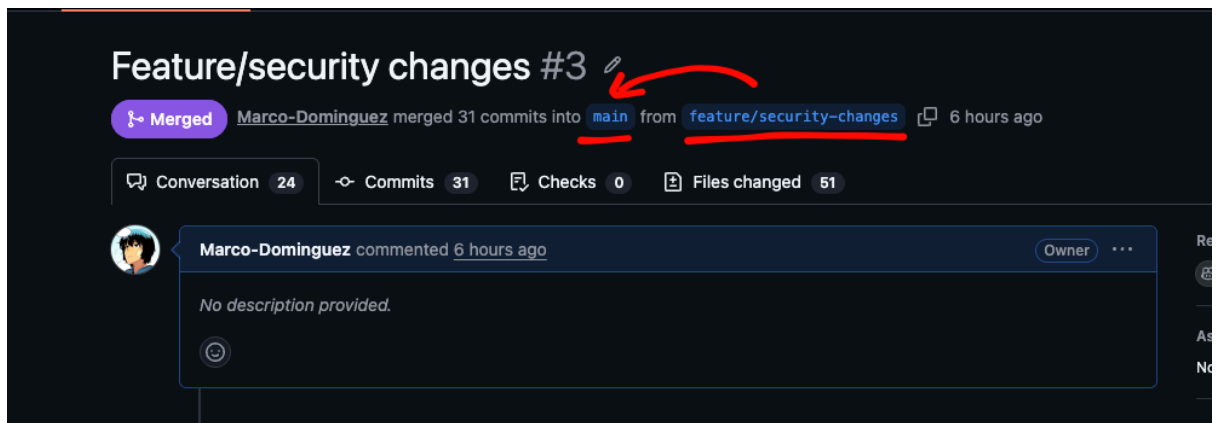
/public_html/. La regla `exclude` es para que las carpetas innecesarias como `.git`, la propia configuración `.github/`, archivos `README.md` y la carpeta `assets/` (carpeta donde subimos los archivos .ai y PDFs dl diseño) no se suban y no consuman espacio en el server.

Parte 2: Explicación del Flujo de Despliegue (Cómo funciona al subirlo)

Una vez configurado todo lo anterior, cada que subamos cambios a la rama o ramas establecidas se ejecutara de la siguiente manera:

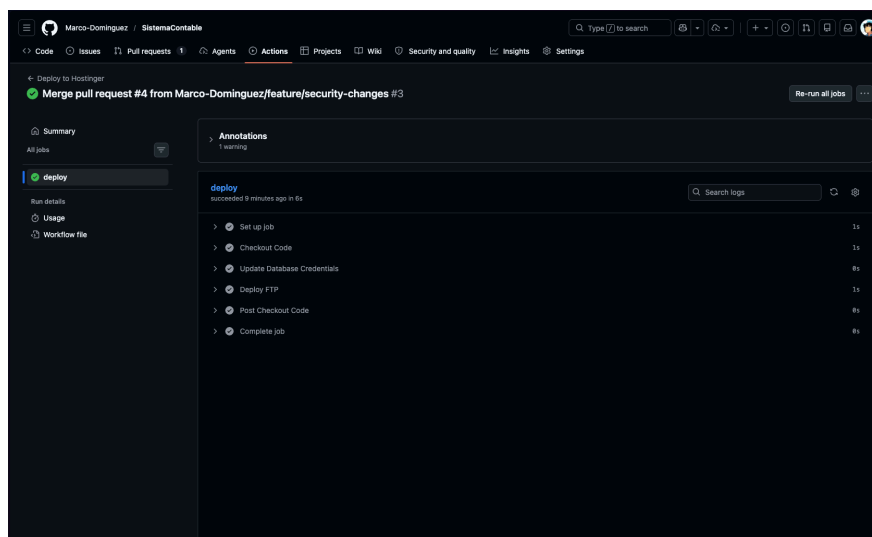
Paso 1: El desarrollador sube los cambios (Push)

Una vez que terminas de programar algo nuevo en tu local, guardas los cambios y corres en la terminal `git push origin main` o se hace un merge desde otra rama a la rama configurada.



Paso 2: GitHub Actions detecta el cambio

GitHub reconoce inmediatamente que ha llegado código nuevo a la rama `main` y lee el archivo `deploy.yml`. Al hacerlo, enciende un servidor virtual para procesar el código de forma aislada.



Paso 3: Transformación del código (Sustitución de Credenciales)

Antes de enviar el código al servidor, GitHub abre el archivo 'database.php'. Toma los secretos que configuramos y los cambia en el archivo. Así, el código deja de apuntar a "XAMPP/localhost" y empieza a apuntar a la DB en el servidor de Hostinger.

```
be > config > database.php
<?php
//configs de la db
define('DB_HOST', 'localhost');
define('DB_PORT', '3306');
define('DB_NAME', 'sistema_contable');
define('DB_USER', 'root');
define('DB_PASS', '');
define('DB_CHARSET', 'utf8mb4');

class Database {
    private static ?PDO $instance = null;

    public static function getInstance(): PDO {
        if (self::$instance === null) {
            $dsn = sprintf(
```

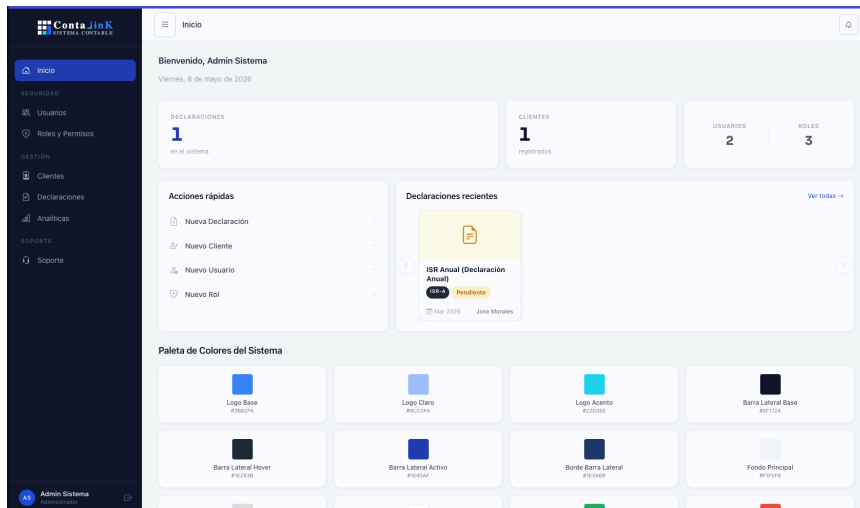
Paso 4: Sincronización FTP

Una vez que se cambian las configuraciones de la DB, GitHub Actions usa las configuraciones del cliente FTP (como si usaras FileZilla). Se conecta a Hostinger, revisa qué archivos son nuevos o han sido modificados, y los manda uno a uno al directorio /public_html/, omitiendo la carpeta assets y los archivos de git.

```
Deploy FTP
22 -----
23 Local Files: 79
24 Server Files: 0
25 -----
26 Calculating differences between client & server
27 -----
28 Create: service
29 Create: service/config
30 Create: service/controllers
31 Create: service/middleware
32 Create: view
33 Create: view/assets
34 Create: view/assets/css
35 Create: view/assets/img
36 Create: view/assets/js
37 Create: view/components
38 Create: view/components/templates
39 Create: view/pages
40 Create: view/pages/analiticas
41 Create: view/pages/auth
42 Create: view/pages/clientes
43 Create: view/pages/clientes/tabs
44 Create: view/pages/dashboard
45 Create: view/pages/declaraciones
46 Create: view/pages/declaraciones/tabs
47 Create: view/pages/security
48 Create: view/pages/security/roles
49 Create: view/pages/security/roles/tabs
50 Create: view/pages/security/seguridad
```

Paso 5: Despliegue Exitoso

La máquina virtual de GitHub se apaga automáticamente tras subir el último archivo. Después de eso ya se pueden ver los cambios reflejados.



Nota: en este despliegue automático se subieron cambios del logo y la paleta de colores. aquí se puede ver que los cambios se subieron correctamente